

Programación Científica · 2026-1

Sistemas Dinámicos

Módulo 4 - Clase 1

¿Puede una ecuación describir cómo cambia una sociedad?

Universidad Nacional de Colombia

`mbastidaso@unal.edu.co`

¿Qué es un sistema dinámico?

Estado y espacio de estados

Estado

El **estado** $x(t)$ es toda la información del sistema en el instante t que determina su evolución futura.

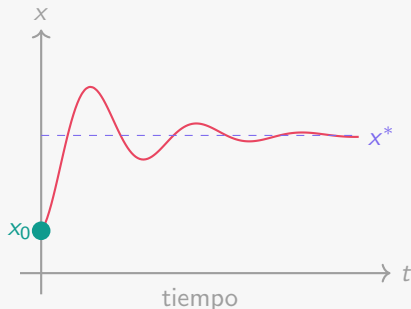
Trayectoria

La **trayectoria** $x : \mathbb{R} \rightarrow \mathcal{X}$ es la curva que recorre el estado a lo largo del tiempo.

Ejemplos:

- Opinión de N personas:
 $x = (x_1, \dots, x_N) \in \mathbb{R}^N$.
- Fase de N osciladores:
 $\theta = (\theta_1, \dots, \theta_N) \in \mathbb{R}^N$.
- Temperatura de un sólido: $T \in \mathbb{R}^+$.

El conjunto de todos los estados posibles es el **espacio de estados** $\mathcal{X}$.



La EDO como lenguaje

Ecuación diferencial ordinaria (EDO)

$$\dot{x}(t) = f(x(t), t), \quad x(t_0) = x_0$$

- $f : \mathcal{X} \times \mathbb{R} \rightarrow \mathcal{X}$ **campo vectorial**: especifica dirección y velocidad de cambio.
- La EDO no dice *dónde está* x ; dice *cómo cambia*.
- x_0 **condición inicial**: fija la trayectoria.

Teorema de Picard-Lindelöf (enunciado informal)

Si f es continua y (local) Lipschitz en x , existe una única solución $x(t)$ en un intervalo $[t_0, t_0 + \delta)$.
El modelo está bien planteado: dado x_0 , la trayectoria queda determinada.

Tipos de sistemas dinámicos

Eje	Opción A	Opción B
Tiempo	Discreto ($t \in \mathbb{Z}$, $x_{n+1} = g(x_n)$)	Continuo ($t \in \mathbb{R}$, EDO $\dot{x} = f(x, t)$)
Autonomía	No autónomo: $f = f(x, t)$	Autónomo: $f = f(x)$
Linealidad	Lineal: $f(x) = Ax$	No lineal: f arbitraria
Incertidumbre	Determinista	Estocástico

Los modelos de este módulo

Hegselmann-Krause: continuo, autónomo, **no lineal**.

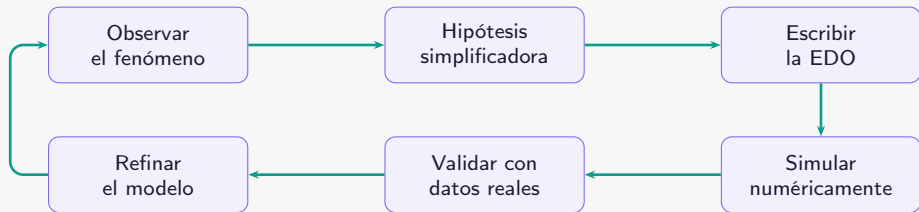
Kuramoto: continuo, autónomo, **no lineal**.

Ambos son deterministas: dada x_0 , la trayectoria es única.



El ciclo de modelación

Del fenómeno a la ecuación



Esta clase

Seguiremos este ciclo en vivo para construir el modelo H-K.

Empezamos con la **observación**: ¿por qué las redes sociales nos polarizan?

Construcción

Modelo de Hegselmann-Krause

¿Por qué internet nos polariza?

Observación: el límite de confianza

Pausa

¿Qué hace que tu opinión cambie?

¿Qué hace que ignores completamente la opinión de alguien?

Hipótesis (Hegselmann & Krause, 2002):

- Cada individuo tiene una opinión $x_i \in [-1, 1]$ (-1 ext. izquierda, $+1$ ext. derecha).
- Solo interactúa con quienes tienen opiniones **cercanas**: $|x_j - x_i| < \varepsilon$.
- Su opinión se mueve hacia el **promedio** de esa vecindad.
- $\varepsilon > 0$ es el **umbral de confianza**.

El papel del algoritmo

Las plataformas reducen el ε efectivo: solo muestran contenido similar al que ya se consume. El modelo captura exactamente este efecto.

Dos agentes: la EDO mínima

Con $N = 2$ individuos, $x_1, x_2 \in [-1, 1]$:

$$\dot{x}_1 = \begin{cases} x_2 - x_1 & \text{si } |x_2 - x_1| < \varepsilon \\ 0 & \text{en otro caso} \end{cases}$$

$$\dot{x}_2 = \begin{cases} x_1 - x_2 & \text{si } |x_2 - x_1| < \varepsilon \\ 0 & \text{en otro caso} \end{cases}$$

Lectura de la EDO

Si están dentro del radio de confianza, cada agente se mueve hacia el otro a velocidad proporcional a la diferencia de opiniones.

Si están lejos: $\dot{x}_i = 0$ (silencio).

Equilibrios

La diagonal $x_1 = x_2$ es un conjunto de equilibrios (consenso).

Si $|x_1(0) - x_2(0)| \geq \varepsilon$: ambas opiniones quedan fijas (disenso permanente).

N agentes: el modelo completo

Modelo de Hegselmann-Krause

$$\dot{x}_i(t) = \frac{1}{|\mathcal{N}_i(t)|} \sum_{j \in \mathcal{N}_i(t)} (x_j(t) - x_i(t)), \quad i = 1, \dots, N,$$

donde la **vecindad de confianza** del agente i es

$$\mathcal{N}_i(t) = \{j : |x_j(t) - x_i(t)| < \varepsilon\}.$$

EDO vectorial

El estado es $x = (x_1, \dots, x_N) \in [-1, 1]^N$.

El campo $f(x)$ es discontinuo: salta cada vez que un par entra o sale de la vecindad.

Parámetro clave

$\varepsilon \rightarrow 0$: nadie escucha = opiniones fijas.

$\varepsilon \rightarrow \infty$: todos se escuchan = consenso.

ε^* intermedio: fragmentación.

¿Por qué no hay solución analítica?

Obstáculos del modelo H-K:

- $\mathcal{N}_i(t)$ **cambia con el tiempo**: el grafo de interacciones evoluciona con la solución misma.
- $f(x)$ es **discontinua**: salta cada vez que $|x_j - x_i|$ cruza ε .
- El sistema **acopla** las N ecuaciones de forma no trivial.

Consecuencia

Para el modelo H-K (y para la mayoría de EDOs no lineales de interés práctico) no existe fórmula cerrada para $x(t)$.

Necesitamos métodos numéricos: aproximar la solución con una secuencia de puntos $x_0, x_1, x_2, \dots$ a tiempos $t_0, t_1, t_2, \dots$

Solución del modelo H-K

El modelo que construimos es de la forma:

$$y'(t) = f(t, y), \quad a \leq t \leq b, \quad y(a) = \alpha.$$

Para H-K con N agentes, $y = (x_1, \dots, x_N)$ es un vector y f depende de qué vecinos están dentro del radio ε .

Por qué no hay solución cerrada

La vecindad $\mathcal{N}_i(t)$ cambia con t : el campo f es discontinuo cada vez que un par entra o sale del radio.

Necesitamos **aproximar** $y(t_i) \approx w_i$ en una malla de puntos $t_0, t_1, \dots, t_N$.

Plan de la clase

1. Existencia y unicidad (Picard).
2. Método de Euler: derivación y cota de error.
3. Error de truncamiento local.
4. Familia de Runge-Kutta hasta orden 4.
5. Consistencia, convergencia, estabilidad.

Existencia y unicidad

¿está bien planteado el problema?

Condición de Lipschitz y unicidad

Definición: condición de Lipschitz

$f(t, y)$ satisface una **condición de Lipschitz** en y sobre $D = \{(t, y) \mid a \leq t \leq b, -\infty < y < \infty\}$ si existe una constante $L > 0$ tal que

$$|f(t, y_1) - f(t, y_2)| \leq L |y_1 - y_2| \quad \forall (t, y_1), (t, y_2) \in D.$$

Teorema de Picard-Lindelöf

Si f es continua en D y satisface la condición de Lipschitz en y , entonces el PVI

$$y'(t) = f(t, y), \quad a \leq t \leq b, \quad y(a) = \alpha,$$

tiene una **solución única** $y(t)$ para $t \in [a, b]$.

Para H-K: si ε está fijo, f es Lipschitz por partes. La unicidad garantiza que la solución numérica aproxima algo bien definido.

Método de Euler

derivación desde Taylor

La malla y la idea de Euler

Paso 1: discretizar el tiempo. Se dividen $[a, b]$ en N subintervalos iguales:

$$t_i = a + ih, \quad h = \frac{b-a}{N}, \quad i = 0, 1, \dots, N.$$

h es el **tamaño del paso**.

Paso 2: expandir en Taylor y truncar. Si $y \in C^2[a, b]$, por Taylor alrededor de t_i :

$$y(t_{i+1}) = y(t_i) + h y'(t_i) + \frac{h^2}{2} y''(\xi_i), \quad \xi_i \in (t_i, t_{i+1}).$$

Reemplazando $y'(t_i) = f(t_i, y(t_i))$ y **descartando** el término $O(h^2)$:

$$y(t_{i+1}) \approx y(t_i) + h f(t_i, y(t_i)).$$

Fórmula del método de Euler

Método de Euler (orden 1)

Dado el PVI $y' = f(t, y)$, $y(a) = \alpha$:

$$w_0 = \alpha,$$

$$w_{i+1} = w_i + h f(t_i, w_i), \quad i = 0, 1, \dots, N-1.$$

w_i es la aproximación numérica de $y(t_i)$.

Cota de error global del método de Euler

Teorema — cota de error de Euler

Sea f continua y Lipschitz con constante L , y sea M tal que $|y''(t)| \leq M$ para todo $t \in [a, b]$. Entonces las aproximaciones $w_0, w_1, \dots, w_N$ de Euler satisfacen:

$$|y(t_i) - w_i| \leq \frac{hM}{2L} \left[e^{L(t_i - a)} - 1 \right], \quad i = 0, 1, \dots, N.$$

Lectura de la cota

El error crece exponencialmente en t (factor $e^{L(t_i - a)}$), pero es **proporcional a h** : reducir el paso a la mitad reduce el error a la mitad.
Orden global: $O(h)$.

Para H-K

$M = \max |y''|$ depende de ε . La cota es útil para elegir un h que garantice el error deseado en el tiempo final b .

Método de Taylor de orden n

Usando los términos de Taylor hasta orden n :

$$y(t_{i+1}) = y(t_i) + h f + \frac{h^2}{2} f' + \frac{h^3}{6} f'' + \dots + \frac{h^n}{n!} f^{(n-1)} + O(h^{n+1}).$$

Método de Taylor de orden n

$$w_{i+1} = w_i + h T^{(n)}(t_i, w_i),$$
$$T^{(n)} = f + \frac{h}{2} f' + \frac{h^2}{6} f'' + \dots + \frac{h^{n-1}}{n!} f^{(n-1)}.$$

Error local $O(h^n)$. Euler = Taylor de orden 1.

El problema de Taylor de orden alto

Requiere calcular $f', f'', \dots$ analíticamente.

Para H-K: f es discontinua $\Rightarrow$ no es diferenciable.

Idea de Runge-Kutta: lograr el mismo orden de precisión *evaluando f múltiples veces*, sin derivarla.

Familia de Runge-Kutta

evaluar f en lugar de derivarla

Runge-Kutta de orden 4 (RK4)

Método de Runge-Kutta de orden 4

$$w_0 = \alpha,$$

$$k_1 = h f(t_i, w_i),$$

$$k_2 = h f\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_1\right),$$

$$k_3 = h f\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right),$$

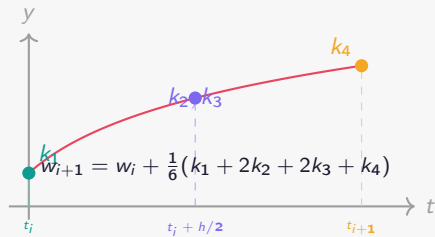
$$k_4 = h f(t_i + h, w_i + k_3),$$

$$w_{i+1} = w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

para $i = 0, 1, \dots, N - 1$.

Geometría de RK4: promediar cuatro pendientes

Cada k_j/h es una pendiente evaluada en un punto distinto del intervalo:



Los pesos 1, 2, 2, 1 corresponden a la cuadratura de Simpson aplicada sobre el intervalo $[t_i, t_{i+1}]$. RK4: error local $O(h^5)$, error global $O(h^4)$, cuatro evaluaciones de f por paso.

Comparación de la familia de métodos

Método	Evaluaciones	Error global	Cuándo usar
Euler	1	$O(h)$	prototipo / referencia
Punto medio	2	$O(h^2)$	problemas simples
Euler modificado	2	$O(h^2)$	igual que punto medio
Heun (orden 3)	3	$O(h^3)$	intermedio
RK4	4	$O(h^4)$	uso general

Regla práctica

Para reducir el error de **Euler** en $10\times$ hay que reducir h en $10\times$ (10 veces más lento).

Para el mismo efecto con **RK4**: reducir h en $10^{1/4} \approx 1.78\times$.

Con $h = 0.1$: error Euler $\sim 10^{-1}$, error RK4 $\sim 10^{-4}$.

Aplicación

RK4 en sistemas vectoriales

el modelo H-K como PVI de orden 1 en $\mathbb{R}^N$

Sistema de EDOs de primer orden

El modelo H-K con N agentes es un **sistema vectorial**:

$$\frac{du_1}{dt} = f_1(t, u_1, \dots, u_m), \quad \dots \quad \frac{du_m}{dt} = f_m(t, u_1, \dots, u_m), \quad u_j(a) = \alpha_j.$$

En notación vectorial: $u' = f(t, u)$, $u(a) = \alpha$.

RK4 vectorial para H-K

Los k_j son ahora **vectores** en $\mathbb{R}^N$:

$$\begin{aligned} k_1 &= h f(t_i, w_i), \\ k_2 &= h f\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_1\right), \\ k_3 &= h f\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right), \\ k_4 &= h f(t_i + h, w_i + k_3), \\ w_{i+1} &= w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

El código es idéntico al escalar: NumPy opera en arrays de cualquier dimensión.